Module Code: BEMM459

Module Name: Database Technologies for Business Analytics

Group J:

Aarathi Jeevan Kumar

Aiswarya V Chandran

Ashwin Varghese John

Devika Girish Varma

George Antony

Table of Contents

A1) Olist - A Brazilian E-commerce company 3

Overview 3

Business Functions 3

Use case scenario 4

Choice of Databases 5

Need for Both RDBMS and NoSQL 5

Why MongoDB was the preferred NoSQL database 7

Why document database over other types of NoSQL databases 8

A2) ER Diagram 9

A3) Data Dictionary 10

Logical Database Design using DBDL 13

A4) Data Storage in NoSQL Database 15

A5) How Olist Implements Polyglot persistence 17

Recommendations 19

Reference 19

A1) Olist - A Brazilian E-commerce company

Overview

Olist is a Brazilian e-commerce platform that links small and medium-sized companies to online marketplaces so they can sell their goods there.

The database system consists of users that are customers as well as sellers. The customers place orders on the products listed by the sellers. They enter the information regarding their address which is put into the database and a customer id is created for them. They pay accordingly whether through installments or directly and information on the type of payments is also stored in the database. The customers can see how the orders are shipped and delivered. The sellers can list their products and see the orders taken for their products by the customers through the database. They can see whether the orders are delivered to the customers on time or not.

Business Functions

Following categories best describe how Olist Store conducts business:

- Sales: Olist provides a platform for small and medium-sized businesses to sell their goods online. This calls for the development of a storefront, the management of products, the processing of orders, the handling of payments, and the control of deliveries.

- Marketing: To assist companies in becoming more visible to consumers and attracting more customers, Olist Store also provides marketing tools. This covers marketing and promotional efforts as well as search engine optimization.

- Customer Relationship Management (CRM): To assist businesses in managing their customer relationships, Olist Store offers a CRM system. This includes managing feedback as well as order tracking and customer service.

- Analytics: Olist Store provides analytics tools to help companies monitor the success of their marketing and sales efforts. Data on sales, consumer behaviour, and advertising campaigns are included.

- Logistics: By connecting them with shipping partners and offering delivery tracking services, Olist Store helps businesses with their logistics needs.

- Olist Store provides businesses with financial services like revenue management and payment processing.

In conclusion, the Olist store's business functions - Sales, CRM, analytics, logistics, and payment - are all created to assist small and medium-sized businesses to efficiently sell their products online.

Use case scenario

The success of Olist Store largely depends on effective management of its products, which can be achieved through the use of a dashboard that provides a range of useful features to help manage products and make data-driven decisions to improve the store's performance and quality.

The products section manages the different products that are brought in by the sellers and could be viewed by the users. The products and their corresponding category along with what quantity of the product is remaining is a main focus of the application.

The application should provide a comprehensive overview of all products currently available on the Olist store, along with the average sale price of each product category. This information can be used to gain valuable insights into the performance of different products, including which products are selling the most, which products are receiving the highest ratings, and which products are receiving the most reviews.

In addition, the application should provide a convenient way to modify product information, including titles, descriptions, images, and prices. Customer reviews of purchased products

should also be easily monitored and managed through the dashboard. In summary, the application should include features such as product overview, sales analytics, product performance metrics, product editing, and review management. These features will help ensure that the Olist Store is able to effectively manage its products and make informed decisions to improve the performance and quality of the store.

Choice of Databases

Need for Both RDBMS and NoSQL

Both RDBMS and NoSQL databases are used for the best outputs of Olist. RDBMS and NoSQL will probably commend one another and adopt features from one another as they are used in a demanding application such as Olist.

Users can build, update, administer, and interact with a relational database using an RDBMS, enabling the tabbing of data. For Olist, using RDBMS as a sophisticated data management system greatly facilitates the process of gaining insights from data. As multiple users and sellers are trying to access the same data at the same time, there needs to be data consistency and relational databases are devised to support these multistep procedures that are treated as a single transaction.

Using RDBMS is crucial because it adheres to a serial process and applies the Atomic, Consistent, Isolated, and Durable (ACID) qualities while performing transaction modifications and queries regardless of any interruption, including a power outage, software malfunction, or operating system breakdown.

- Atomicity: This refers to the concept that a transaction (a single query or a series of operations) is an intact part that can either be completed in its whole or not at all. It's an “All or nothing” property.

- Consistency: this means that a transaction must transform the database from one consistent state to another consistent state. A transaction performed in memory, for example, must be committed to secondary storage in order for it to be permanently

stored. The last consistent state would have been the one that existed before the transaction(s) were completed, if the commit command had not been given.

- Isolation: this refers to the fact that transactions are carried out separately from one another and that unfinished transactions are hidden from other transactions.

- Durability: This describes the fact that a successfully completed (committed) transaction's consequences are permanently stored in the database and cannot be lost in the event of a failure or interruption.

Consistency, Availability and Partition Tolerance are three proprieties that are required while we use a database. The CAP Theorem (also known as Brewer's Theorem) states that Distributed databases cannot have consistency (C), availability (A), and partition protection (P) all at the same time.

- Consistency: The most recent information is provided with each read or query.

- Availability: There is an answer to every request (without guarantee of being the most up to date)

- Partition: Even though there is a communication breakdown between the servers, the system still functions.

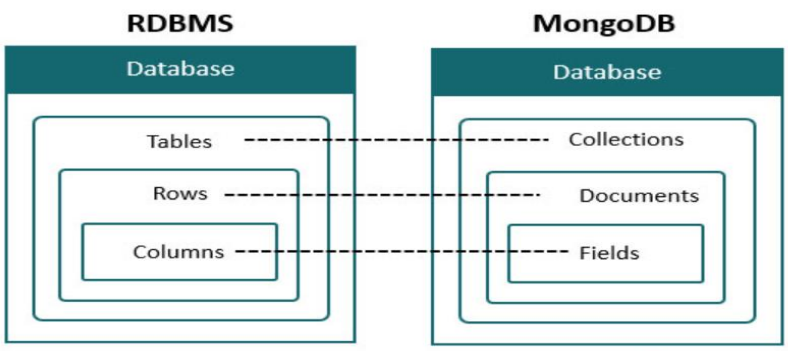
Since the consistency of the database in RDBMS takes precedence over availability, Olist needs RDBMS for essential transactions. For example, for each product listed by the seller the application shows updated information regarding the quantity of product that is remaining for the user to place their orders accordingly. If the data is not consistent, the customer could place orders on products that are not available therefore losing money. For a seller that is using Olist, the most recent updates on the shipping information regarding orders that have been placed needs to be seen so that they can make sure that no misplacing of orders will take place.

For an application such as Olist, Scalability plays a huge part. We might need to add servers as required so that the application could grow and for this, we need NoSQL databases. NoSQL databases can scale out by utilising commodity hardware rather than scaling up by adding more servers. With no downtime, this has the capacity to sustain greater traffic in order to meet demand. NoSQL databases have taken over as the go-to solution for dynamic data sets because they can scale out to become bigger and more potent.

Data is automatically replicated over numerous servers, data centres, or cloud resources via NoSQL databases. This reduces latency for users, regardless of their location. Also, by lightening the load of database management, this functionality allows for more time to be dedicated to other priorities.

Using NoSQL is quick and easy, but offers little to no structure to impose restrictions on the data. Using RDBMS fulfils each ACID requirement, keeping your data secure and uncontaminated. However, performance declines quickly as traffic and data set size increase. Therefore as a solution, Olist uses an RDBMS to keep data coherent and keep complex queries possible. Operations on the applications are possible as long as the NoSQL layer is able to block the majority of traffic going to the RDBMS. Several techniques exist to accomplish so utilising a NoSQL solution: caching denormalized entities (in a key-value store, index by ID the content of previously connected tables), hosting all derived data (like precalculated recommendation matrices) keeping all the transient, dynamic info on hand (like user sessions, shopping carts)

We could also say that why Olist must combine RDBMS and NoSQL databases is the complexity, risk, and cost of database replication implementation.



Why MongoDB was the preferred NoSQL database

For Olist application, we have used a NoSQL document database, MongoDB.

The main requirements of the Olist application are:

- Ensure constant data accessibility
- Keep the polling rate high when the usage is high
- Save large amounts of information
- Dynamically and continuously convey information about changes (such as the availability of specific products from the product catalogue)

This is mostly important during the peak sales season when there might be an increased number of queries. Because of this using MongoDB as a database is preferred by Olist because of the database scalability.

MongoDB is a document database designed for simple scaling and creation. BSON (Binary JSON) is the format used for creating and storing documents. The conversion of the queries and results into a format that "understands" the front-end code of the Olist application is made very simple by the usage of JSON. Also, it is easier for people to understand. For improved scalability and high availability, the NoSQL solution incorporates hierarchy, automatic fragmentation, and built-in replication.

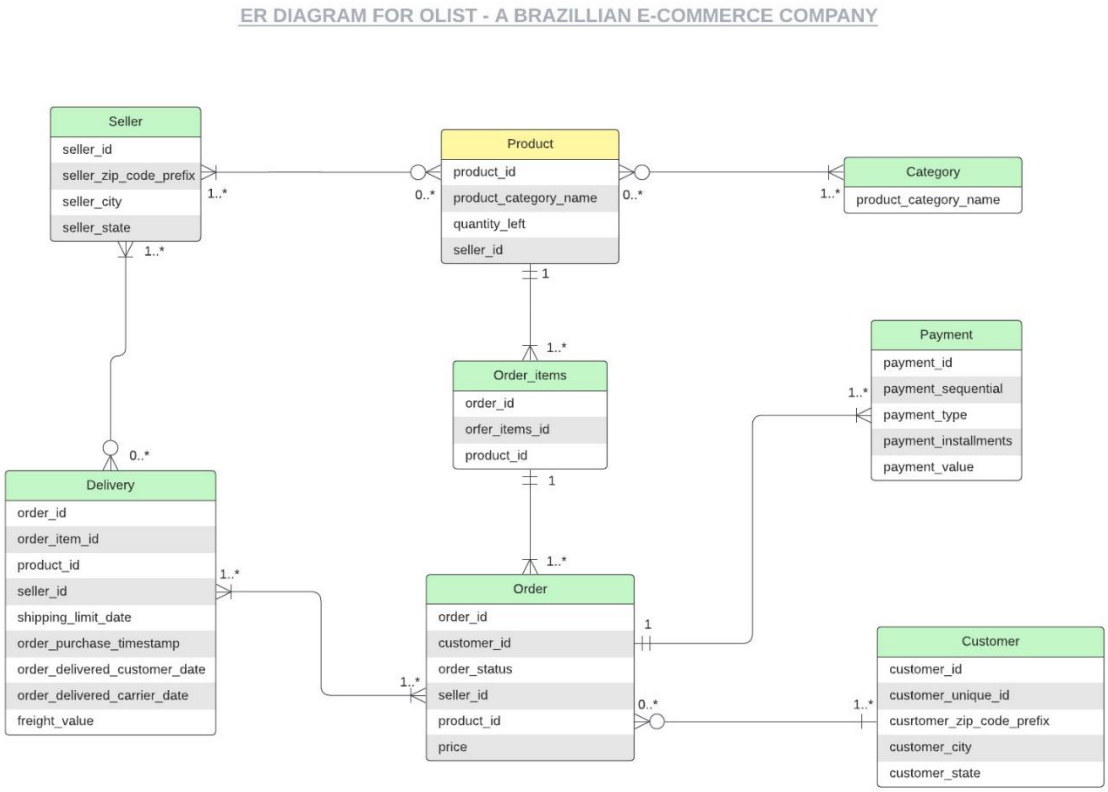
Why document database over other types of NoSQL databases

Developers can use tricks, such as enumerated keys, to implement range queries, but key value databases usually lack the query capabilities of document databases which are required for Olist Application.

Column Family database management systems run on clusters of multiple servers. A column family database is definitely more than what we needed because of this.

Consider applications, where relationship between entities such as highways connecting cities, proteins interacting with other proteins, and employees working with other employees. In all these cases, there is some type of connection, link, or direct relationship between two instances of entities. In this case we use a graph database. For Olist, two orders probably have no connection to each other. They might be ordered by the same customer, but that is a shared attribute, not a connection. Therefore, we can say that for an e-commerce application such as Olist, using MongoDB as a NoSQL database is the best option.

A2) ER Diagram



A3) Data Dictionary

Entity	Attribute Name	Description	Domain attribute /data type	Nulls	Multi - valued
customers_details	customer_id	Unique identifier for a customer	Text (50)	No	No
	customer_unique_id	Unique identifier for each customer	Text (50)	No	No
	customer_zip_code_prefix	Postal code prefix of the customer's address	Text (10)	No	No
	customer_city	City where customer resides	Text (50)	No	No
	customer_state	State where customer resides	Text (10)	No	No
order_items_details	order_id	Unique identifier for the order	Text (50)	No	No
	order_item_id	Unique identifier for an item within an order	Text (50)	No	No
	product_id	Unique identifier	Text (50)	No	No

		for the product sold			
	shipping_limit_date	Date and time by which the seller must ship the product	Date	No	No
	price	Price of the product ordered	Float	No	No
	freight_value	Cost of shipping the product ordered	Float	No	No
seller_details	seller_id	Unique identifier for a seller	Text (50)	No	No
	seller_zip_code_prefix	Postal code prefix of seller's address	Text (10)	No	No
	seller_city	City where the seller is located	Text (50)	No	No
	seller_state	State where the seller is located	Text (10)	No	No
orders_details	order_id	Unique identifier for an order	Text (50)	No	No
	customer_id	Unique identifier for the customer	Text (50)	No	No

		who placed the order			
	seller_id	Unique identifier for the seller of the product	Text (50)	No	No
	order_status	Status of the order	Text (50)	No	No
	order_purchase_date	Date when the order was placed	Date	No	No
	order_delivered_carrier_date	Date when the order was delivered to the carrier	Date	No	No
	order_delivered_customer_date	Date when the order was delivered to the customer	Date	No	No
order_payments_details	order_id	Unique identifier for the order	Text (50)	No	No
	payment_sequential	Sequence number of the payment	Integer	No	No
	payment_type	Mode of payment	Text (50)	No	No

	payment_installments	Number of installments for the payment	Integer	No	No
	payment_value	Value paid	Float	No	No
products_details	product_id	Unique identifier for a product	Text (50)	No	No
	seller_id	Unique identifier for the seller of the product	Text (50)	No	No
	product_category_name	Category of the product	Text (50)	No	No
	quantity_remaining	Quantity remaining in stock	Integer	No	No

Logical Database Design using DBDL

Table 1 : customers_details Table

customers_details(customer_id,customer_unique_id,customer_zip_code_prefix,customer_city,customer_state)

Primary Key : customer_id

Table 2: order_items_details Table

order_items_details(order_id, order_item_id, product_id, shipping_limit_date, price, freight_value)

Primary Key : order_item_id
Foreign Key : order_id, product_id

Table 3: sellers_details Table

sellers_details (seller_id, seller_zip_code_prefix, seller_city, seller_state)
Primary Key : seller_id

Table 4: products_details Table

products_details(product_id, seller_id, product_category_name, quantity_remaining)

Primary Key : product_id
Foreign Key : seller_id

Table 5 : orders_details Table

orders_details(order_id, customer_id, seller_id, order_status, order_purchase_date, order_delivered_carrier_date, order_delivered_customer_date)

Primary Key : order_id
Foreign Key : customer_id, seller_id

Table 6 : order_payments_details Table

order_payments(order_id, payment_sequential, payment_type, payment_installments, payment_value)

Primary Key : payment_sequential

Foreign Key : order_id

A4) Data Storage in NoSQL Database

Data in MongoDB is kept in collections, which are comparable to tables in RDBMS. Each record is kept in a document within the collection, which contains many documents. In contrast to RDBMS, which stores data within a table in rows and columns, JSON stores data in key-value pairs (key: value).

To effectively store product descriptions and reviews in MongoDB, we established two separate collections: one for product descriptions and another for reviews. Within the product descriptions collection, we store pertinent details for each product, such as its product id, product category name, product description, product photos quantity, product weight in grams, product length in cm, product height in cm and product width in cm. Each document within this collection represents a singular product.

The reviews collection houses details about customer review. Within the reviews collection, we have review id, corresponding order id, the review score on a scale of 1 to 5, review comment message, review creation date. Each document within this collection is stored in key value pairs. To link reviews to their corresponding orders, we've included an Order ID field within each review document.

As we know that MongoDB databases are schemaless. So, it is not necessary in a collection that the schema of one document is similar to another document. Or in other words, a single collection contains different types of documents like as shown in the below example where product description collection contain two different types of documents.

Pseudocode used for a document in product description collection (two BSON files):

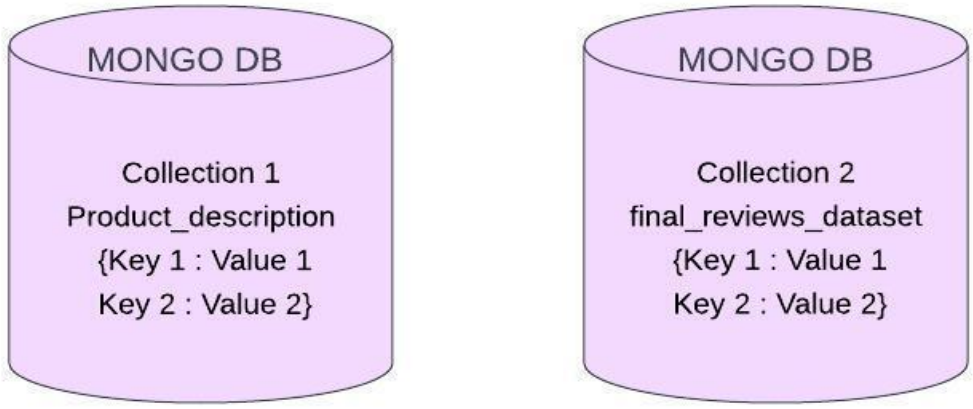
```
{  "product_id": "16ba44d9231d19fc57e0a4188d0ce1cf",
   "product_category_name": "furniture_decor",
   "Product Description": "Furniture and home decoration products.",
   "product_photos_qty": 1,
```

"product_weight_g": 303,
"product_length_cm": 25,
"product_height_cm": 2,
"product_width_cm": 17
}

{
"product_id": "0602bc6086443c4e9b9e5ba88119ce08",
"product_category_name": "perfumery",
"Product Description": "Perfumes and fragrances",
"product_photos_qty": 1,
"product_weight_g": 133,
"product_length_cm": 21,
"product_height_cm": 12,
"product_width_cm": 19
}

Pseudocode used for a document in reviews collection (One BSON File):

{
"review_id": "fc35cf08ab23f83e759a28af9e5953fd",
"order_id": "b73f09200030e4970b9840b72a630e45",
"review_score": 5,
"review_comment_message": "Fast delivery!",
"review_creation_date": "2/15/2017 0:00"
}



A5) How Olist Implements Polyglot persistence:

Polyglot persistence refers to a design approach in which multiple data storage technologies are employed within a single application, selected based on the specific characteristics and intended manipulation of the data. This approach was developed after acknowledging that varying types of data have unique storage and processing requirements.

Olist being an e-commerce platform has access to large varieties of data allowing for polyglot persistence to be implemented in several ways. One possible approach that we have used here is as follows:

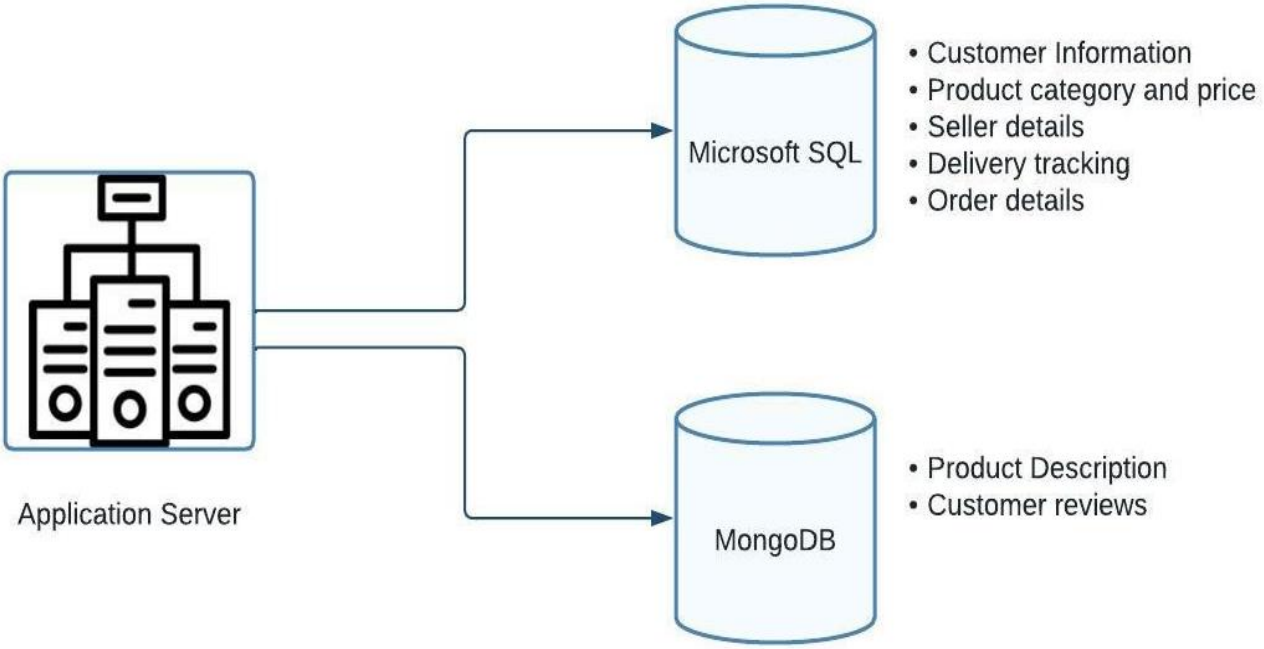
SQL databases are well-suited for structured data that fits neatly into tables with predefined relationships. They are also fitting for financial data like payment information since they offer ACID compliance, ensuring transactions are reliably processed and data is consistent.

Microsoft SQL Server manages our products, order transactions, customer data, and financial records. We use MongoDB to store product descriptions and customer reviews. This approach can help us achieve optimal performance, scalability, and maintainability for our application.

For our application, we have established polyglot persistence by creating a link for our products which is part of RDBMS and corresponding customer review which is part of NoSQL database.

Additionally, it allows us to use the right tool for the right job, instead of trying to force-fit all data into a single database management system. NoSQL databases have a flexible schema, making it easier to accommodate unstructured data and can handle a high volume of writes and reads.

Overall, polyglot persistence can help Olist build a flexible and scalable application that can handle a wide variety of data storage needs while avoiding the limitations of a single database model.



Recommendations

- Recommendation Engine: Suggests products to customers based on their browsing and purchase history which could increase sales and improve customer satisfaction.
- Sentiment Analysis: Help identify areas of improvement for the platform, as well as help sellers understand their customers' needs and preferences by analyzing customer reviews.
- View and Click Data Analysis: Use this data to understand which products people spend the most time on and send out advertisements online to nudge them to make the purchase.
- Other Platforms Integration: Integrating with other platforms such as social media, search engines, and mobile applications to increase visibility and reach new customers.
- Own Label Business: Analyze user data and purchase patterns to make tailor made products that satisfy most customers and thereby result in a possible sale volume that is more than the market leader in the segment.

Reference

- Sullivan, D. (2015). *NoSQL for mere mortals*. Addison-Wesley.
- MongoDB.(2023).Install MongoDb Community Edition.https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-windows/
- Web Dev Simplified.(2021,September 28).MongoDB Crash Course[Video].YouTube.https://www.youtube.com/watch?v=ofme2o29ngU
- EDUCBA.What is MongoDB?. https://www.educba.com/what-is-mongodb/?source=leftnav